

MODULE 23

Advanced Computer Vision

AI with Data Science - Professional Learning Series

ARSHITH LMS | Professional Edition

Advanced Computer Vision

Computer Vision Overview

Computer Vision enables machines to interpret visual data.

Key tasks:

Image Classification: what is in the image?

Object Detection: where are objects?

Image Segmentation: pixel-level classification

Pose Estimation: body keypoints

Optical Flow: motion detection

Depth Estimation: 3D from 2D

Tools: OpenCV, TensorFlow, PyTorch, YOLO, Detectron2

OpenCV Basics

```
import cv2
import numpy as np

# Read and display image
img = cv2.imread('image.jpg')
img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

# Resize
img_resized = cv2.resize(img, (224, 224))

# Grayscale
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

# Edge detection
blurred = cv2.GaussianBlur(gray, (5,5), 0)
edges = cv2.Canny(blurred, 50, 150)

# Face detection with Haar cascade
face_cascade = cv2.CascadeClassifier(cv2.data.harcascades +
'haarcascade_frontalface_default.xml')
faces = face_cascade.detectMultiScale(gray, 1.1, 4)
```

Image Segmentation

```
import tensorflow as tf
from tensorflow.keras.applications import MobileNetV2

# Semantic segmentation with U-Net
def unet(input_size=(256,256,3), num_classes=21):
    inputs = tf.keras.Input(input_size)
    # Encoder
```

```

c1 = tf.keras.layers.Conv2D(64, (3,3), activation='relu', padding='same')(inputs)
p1 = tf.keras.layers.MaxPooling2D()(c1)
# Bottleneck
bn = tf.keras.layers.Conv2D(128, (3,3), activation='relu', padding='same')(p1)
# Decoder
u1 = tf.keras.layers.UpSampling2D()(bn)
cat1 = tf.keras.layers.Concatenate()([u1, c1])
outputs = tf.keras.layers.Conv2D(num_classes, (1,1), activation='softmax')(cat1)
return tf.keras.Model(inputs, outputs)

```

Real-time Object Detection YOLO

```

from ultralytics import YOLO
import cv2

# Load YOLOv8 model
model = YOLO('yolov8n.pt')

# Inference on image
results = model('image.jpg', conf=0.5)
results[0].show()

# Real-time webcam detection
cap = cv2.VideoCapture(0)
while cap.isOpened():
    ret, frame = cap.read()
    if not ret: break
    results = model(frame)
    annotated = results[0].plot()
    cv2.imshow('YOLO Detection', annotated)
    if cv2.waitKey(1) & 0xFF == ord('q'): break
cap.release()
cv2.destroyAllWindows()

```

GANs for Image Generation

```

import tensorflow as tf

# Generator
def build_generator(latent_dim=100):
    model = tf.keras.Sequential([
        tf.keras.layers.Dense(7*7*256, input_dim=latent_dim),
        tf.keras.layers.Reshape((7, 7, 256)),
        tf.keras.layers.Conv2DTranspose(128, (5,5), strides=2, padding='same',
            activation='relu'),
        tf.keras.layers.Conv2DTranspose(64, (5,5), strides=2, padding='same',
            activation='relu'),
        tf.keras.layers.Conv2DTranspose(1, (5,5), padding='same', activation='tanh')
    ])
    return model

# Discriminator
def build_discriminator():

```

```
model = tf.keras.Sequential([
    tf.keras.layers.Conv2D(64, (5,5), strides=2, padding='same', input_shape=(28,28,1)),
    tf.keras.layers.LeakyReLU(0.2),
    tf.keras.layers.Flatten(),
    tf.keras.layers.Dense(1, activation='sigmoid')
])
return model
```